

Lab03_jhuan

Author

Jhuan Marcos Nunes Solis Silva

Pacotes utilizados e fix para abrir o pdf

```
Sys.setenv(CHROMOTE_EXTRA_ARGS = "--no-sandbox")
Sys.setenv(CHROMOTE_ARGS = "--no-sandbox")
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(leaflet)
```

1

Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.

Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE;

Para o arquivo de vôos:

- Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY;
- Leia apenas 1 milhão de linhas por vez;
- Remova vôos em que o aeroporto de destino comece com a letra 1;
- Remova registros em que existam pelo menos uma coluna faltante;
- Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
- Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

#1

```
airlines <- read_csv("airlines.csv")
```

```
Rows: 14 Columns: 2
```

```
— Column specification —————
```

```
Delimiter: ","
```

```
chr (2): IATA_CODE, AIRLINE
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
#2
```

```
# Importando apenas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE
```

```
airports <- read_csv("airports.csv",  
  col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE))
```

```
Rows: 322 Columns: 5
```

```
— Column specification —————
```

```
Delimiter: ","
```

```
chr (3): IATA_CODE, CITY, STATE
```

```
dbl (2): LATITUDE, LONGITUDE
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
#3
```

```
# Função de processamento
```

```
processa_voos <- function(chunk, pos) {
```

```
  # Remova vôos em que o aeroporto de destino comece com a letra 1
```

```
  chunk_processado <- chunk |>
```

```
    filter(!grepl("^1", DESTINATION_AIRPORT)) |>
```

```
  # Remova registros em que existam pelo menos uma coluna faltante;
```

```
  na.omit() |>
```

```
  # Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do  
  atraso médio por aeroporto de destino
```

```
  group_by(DESTINATION_AIRPORT) |>
```

```
    summarise(
```

```
      soma_atraso = sum(ARRIVAL_DELAY),
```

```
      n_voos = n(),
```

```
      .groups = "drop"
```

```
    )
```

```
  return(chunk_processado)
```

```
}
```

```
# Leitura de flights
```

```
flights_parcial <- read_csv_chunked("flights.csv",  
  callback = DataFrameCallback$new(processa_voos),  
  chunk_size = 1000000,  
  col_types = cols_only(  
    DESTINATION_AIRPORT = col_character(),  
    ARRIVAL_DELAY = col_double()  
  )  
)
```

Ao finalizar, combina as estatísticas para produzir a média final de atraso por aeroporto

```
flights <- flights_parcial |>
  group_by(DESTINATION_AIRPORT) |>
  summarise(
    media_atraso = sum(soma_atraso) / sum(n_voos),
    .groups = "drop"
  )
```

—

2

Selecione a operação apropriada join para incluir, na tabela flights, as colunas CITY e STATE do objeto airports. Para executar esta tarefa: a. Identifique a coluna que é a chave na tabela flights; b. Identifique a coluna que é a chave na tabela airports; c. Quais são os aeroportos que estão listados em flights, mas estão ausentes em airports? d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves; e. Armazene a tabela resultante no objeto flights.

a e b)

A coluna chave de flight é a coluna DESTINATION_AIRPORT e a key de airports é IATA_CODE

c)

```
flights |> anti_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
```

—

A tibble: 0 × 2

i 2 variables: DESTINATION_AIRPORT <chr>, media_atraso <dbl>

d e e)

```
flights <- flights |> left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE"))
```

—

3

Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
airports |> group_by(STATE) |>
  summarise(
    n_linhas = n()
  ) |>
  arrange(desc(n_linhas))
```

—

A tibble: 54 × 2

	STATE	n_linhas
	<chr>	<int>
1	TX	24
2	CA	22

```
3 AK      19
4 FL      17
5 MI      15
6 NY      14
7 CO      10
8 MN       8
9 MT       8
10 NC      8
# i 44 more rows
```

4

Apresente um mapa representando todos os atrasos observados por aeroporto. a. Carregue o pacote leaflet; b. Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico; c. Armazene o gráfico em b) numa variável chamada this; d. Adicione as duas linhas abaixo após a criação da variável this (apenas se você estiver usando Jupyter):

```
# a) feita no inicio do programa
```

```
# b) e c
```

```
this <- flights |>
  leaflet() |>
  addTiles() |>
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    popup = ~DESTINATION_AIRPORT
  )
```

Warning in validateCoords(lng, lat, funcName): Data contains 3 rows with either missing or invalid lat/lon values and will be ignored

```
this
```

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```



```
[1] "2026-08-28 15:13:22 -03"
```